

```
#23070521098 - Nidhi Mangtani
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.applications import VGG16, ResNet50, MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping

# Set random seeds for reproducible results
tf.random.set_seed(42)
np.random.seed(42)

print('TensorFlow Version:', tf.__version__)
```

TensorFlow Version: 2.21.0

```
BASE_DIR = 'Dog Breed Classification'
TRAIN_DIR = os.path.join(BASE_DIR, 'train')
VAL_DIR = os.path.join(BASE_DIR, 'val')
TEST_DIR = os.path.join(BASE_DIR, 'test')

# 5 Dog Breed classes selected for classification
CLASSES = ['beagle', 'bernese_mountain_dog', 'golden_retriever', 'pug', 'rottweiler']
NUM_CLASSES = len(CLASSES)
IMG_SIZE = (224, 224)
BATCH_SIZE = 32

print(f'Selected {NUM_CLASSES} Dog Breed Classes:')
for breed in CLASSES:
    print(f' - {breed}')

# Data Augmentation for training set
train_datagen = ImageDataGenerator(
    rescale=1.0/255.0,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)

# Rescaling for validation and test sets
val_datagen = ImageDataGenerator(rescale=1.0/255.0)
test_datagen = ImageDataGenerator(rescale=1.0/255.0)

# Flow from directory generators for 5 classes
train_generator = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    classes=CLASSES,
    shuffle=True
)

val_generator = val_datagen.flow_from_directory(
    VAL_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    classes=CLASSES,
    shuffle=False
)

test_generator = test_datagen.flow_from_directory(
    TEST_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    classes=CLASSES,
    shuffle=False
)
```

Selected 5 Dog Breed Classes:
- beagle

```
- bernese_mountain_dog
- golden_retriever
- pug
- rottweiler
Found 363 images belonging to 5 classes.
Found 43 images belonging to 5 classes.
Found 50 images belonging to 5 classes.
```

```
def build_transfer_model(model_name, num_classes=5, input_shape=(224, 224, 3)):
    """
    Loads pretrained architecture, removes top 1000-class layer,
    and attaches a new 5-class classification head.
    """
    if model_name == 'VGG16':
        base_model = VGG16(weights='imagenet', include_top=False, input_shape=input_shape)
    elif model_name == 'ResNet50':
        base_model = ResNet50(weights='imagenet', include_top=False, input_shape=input_shape)
    elif model_name == 'MobileNetV2':
        base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=input_shape)
    else:
        raise ValueError(f'Unsupported architecture: {model_name}')

    # Freeze base model convolutional layers initially
    for layer in base_model.layers:
        layer.trainable = False

    # Attach custom classification head (converting 1000-class output to 5-class output)
    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(128, activation='relu')(x)
    x = Dropout(0.3)(x)
    outputs = Dense(num_classes, activation='softmax', name='5_class_output')(x)

    model = Model(inputs=base_model.input, outputs=outputs, name=model_name)
    model.compile(optimizer=Adam(learning_rate=1e-3), loss='categorical_crossentropy', metrics=['accuracy'])
    return model, base_model

def unfreeze_and_compile(model, base_model, num_unfreeze_layers=10):
    """
    Unfreezes the last N layers of base model for fine-tuning with a lower learning rate.
    """
    base_model.trainable = True
    for layer in base_model.layers[:-num_unfreeze_layers]:
        layer.trainable = False
    for layer in base_model.layers[-num_unfreeze_layers:]:
        layer.trainable = True

    model.compile(optimizer=Adam(learning_rate=1e-5), loss='categorical_crossentropy', metrics=['accuracy'])
    return model
```

```
model_names = ['VGG16', 'ResNet50', 'MobileNetV2']
results = {}

for name in model_names:
    print(f'\n=====')
    print(f'    STARTING TRAINING FOR: {name}')
    print(f'=====')

    # Step 1: Build model & convert 1000-class head to 5-class head
    model, base_model = build_transfer_model(name, num_classes=NUM_CLASSES, input_shape=(224, 224, 3))

    # Step 2: Phase 1 Training - Base Frozen (Feature Extraction)
    print(f'--> Phase 1: Training feature extraction head (Base Frozen)...')
    history_p1 = model.fit(
        train_generator,
        validation_data=val_generator,
        epochs=5,
        callbacks=[EarlyStopping(monitor="val_loss", patience=2, restore_best_weights=True)],
        verbose=1
    )

    # Step 3: Phase 2 Fine-Tuning - Unfreeze Last 10 Layers
    print(f'--> Phase 2: Fine-Tuning (Unfreezing last layers)...')
    model = unfreeze_and_compile(model, base_model, num_unfreeze_layers=10)
    history_p2 = model.fit(
        train_generator,
        validation_data=val_generator,
        epochs=3,
        callbacks=[EarlyStopping(monitor="val_loss", patience=2, restore_best_weights=True)],
        verbose=1
    )
```

```
# Step 4: Evaluate on Test Set
test_loss, test_acc = model.evaluate(test_generator, verbose=0)
print(f'🚀 [{name}] Final Test Accuracy: {test_acc*100:.2f}% | Test Loss: {test_loss:.4f}')

# Save metrics and epoch trajectory
results[name] = {
    'model': model,
    'test_acc': test_acc,
    'test_loss': test_loss,
    'val_acc': history_p1.history['val_accuracy'] + history_p2.history['val_accuracy'],
    'val_loss': history_p1.history['val_loss'] + history_p2.history['val_loss'],
    'train_acc': history_p1.history['accuracy'] + history_p2.history['accuracy'],
    'train_loss': history_p1.history['loss'] + history_p2.history['loss']
}
```

```
=====
STARTING TRAINING FOR: VGG16
=====
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN ar
--> Phase 1: Training feature extraction head (Base Frozen)...
Epoch 1/5

1/12 ----- 2:08 12s/step - accuracy: 0.2500 - loss: 1.7110
2/12 ----- 1:06 7s/step - accuracy: 0.1719 - loss: 1.7696
3/12 ----- 58s 6s/step - accuracy: 0.1771 - loss: 1.7115
4/12 ----- 50s 6s/step - accuracy: 0.2031 - loss: 1.6847
5/12 ----- 44s 6s/step - accuracy: 0.1813 - loss: 1.7048
6/12 ----- 38s 6s/step - accuracy: 0.1979 - loss: 1.6671
7/12 ----- 32s 6s/step - accuracy: 0.1920 - loss: 1.6584
8/12 ----- 25s 6s/step - accuracy: 0.1953 - loss: 1.6671
9/12 ----- 19s 6s/step - accuracy: 0.1910 - loss: 1.6732
10/12 ----- 12s 6s/step - accuracy: 0.1813 - loss: 1.6899
11/12 ----- 5s 6s/step - accuracy: 0.1903 - loss: 1.6796
12/12 ----- 0s 6s/step - accuracy: 0.1956 - loss: 1.6919
12/12 ----- 89s 7s/step - accuracy: 0.1956 - loss: 1.6919 - val_accuracy: 0.3023 - val_loss: 1.5521
Epoch 2/5

1/12 ----- 1:25 8s/step - accuracy: 0.2188 - loss: 1.5534
2/12 ----- 1:04 6s/step - accuracy: 0.2500 - loss: 1.5897
3/12 ----- 57s 6s/step - accuracy: 0.2812 - loss: 1.5884
4/12 ----- 50s 6s/step - accuracy: 0.2422 - loss: 1.6362
5/12 ----- 37s 5s/step - accuracy: 0.2374 - loss: 1.6210
6/12 ----- 32s 5s/step - accuracy: 0.2515 - loss: 1.6134
7/12 ----- 27s 6s/step - accuracy: 0.2414 - loss: 1.6097
8/12 ----- 21s 5s/step - accuracy: 0.2383 - loss: 1.6006
9/12 ----- 15s 5s/step - accuracy: 0.2322 - loss: 1.6109
10/12 ----- 10s 5s/step - accuracy: 0.2241 - loss: 1.6133
11/12 ----- 4s 5s/step - accuracy: 0.2387 - loss: 1.6064
12/12 ----- 0s 5s/step - accuracy: 0.2397 - loss: 1.6011
12/12 ----- 65s 5s/step - accuracy: 0.2397 - loss: 1.6011 - val_accuracy: 0.4884 - val_loss: 1.4934
Epoch 3/5

1/12 ----- 38s 3s/step - accuracy: 0.3750 - loss: 1.6001
2/12 ----- 32s 3s/step - accuracy: 0.4219 - loss: 1.5101
3/12 ----- 28s 3s/step - accuracy: 0.3958 - loss: 1.5319
4/12 ----- 25s 3s/step - accuracy: 0.3906 - loss: 1.5184
5/12 ----- 22s 3s/step - accuracy: 0.3875 - loss: 1.5111
6/12 ----- 19s 3s/step - accuracy: 0.3906 - loss: 1.5068
7/12 ----- 16s 3s/step - accuracy: 0.4062 - loss: 1.4915
8/12 ----- 13s 3s/step - accuracy: 0.3828 - loss: 1.5032
9/12 ----- 9s 3s/step - accuracy: 0.3750 - loss: 1.5082
10/12 ----- 6s 3s/step - accuracy: 0.3746 - loss: 1.5079
11/12 ----- 3s 3s/step - accuracy: 0.3837 - loss: 1.5019
12/12 ----- 0s 3s/step - accuracy: 0.3802 - loss: 1.4997
12/12 ----- 43s 4s/step - accuracy: 0.3802 - loss: 1.4997 - val_accuracy: 0.4419 - val_loss: 1.4461
Epoch 4/5

1/12 ----- 40s 4s/step - accuracy: 0.4688 - loss: 1.3767
2/12 ----- 34s 3s/step - accuracy: 0.4531 - loss: 1.4039
3/12 ----- 30s 3s/step - accuracy: 0.4167 - loss: 1.4261
4/12 ----- 26s 3s/step - accuracy: 0.4219 - loss: 1.4118
```

```
summary_list = []
for name, res in results.items():
    summary_list.append({
        'Model Architecture': name,
        'Test Accuracy (%)': round(res['test_acc'] * 100, 2),
        'Test Loss': round(res['test_loss'], 4),
        'Final Val Accuracy (%)': round(res['val_acc'][-1] * 100, 2)
    })

comparison_df = pd.DataFrame(summary_list)
print('\n===== MODEL COMPARISON TABLE =====')
print(comparison_df.to_string(index=False))
```

```

# Comparative Loss & Accuracy Plot
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot Validation Accuracy Curves
for name, res in results.items():
    epochs = range(1, len(res['val_acc']) + 1)
    axes[0].plot(epochs, res['val_acc'], marker='o', linewidth=2, label=f'{name} Val Acc')

axes[0].set_title('Validation Accuracy Comparison', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Epochs (Phase 1 + Phase 2)')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(True, linestyle='--', alpha=0.6)

# Plot Validation Loss Curves
for name, res in results.items():
    epochs = range(1, len(res['val_loss']) + 1)
    axes[1].plot(epochs, res['val_loss'], marker='s', linewidth=2, label=f'{name} Val Loss')

axes[1].set_title('Validation Loss Comparison', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Epochs (Phase 1 + Phase 2)')
axes[1].set_ylabel('Loss')
axes[1].legend()
axes[1].grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()

# Bar Chart for Test Accuracy Comparison
plt.figure(figsize=(8, 4))
bars = plt.bar(comparison_df['Model Architecture'], comparison_df['Test Accuracy (%)'], color=['#3498db', '#e74c3c', '#2ecc71'])
plt.title('Test Accuracy Comparison Across Pretrained CNNs', fontsize=12, fontweight='bold')
plt.xlabel('Model Architecture')
plt.ylabel('Test Accuracy (%)')
plt.ylim(0, 100)

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2.0, height + 2, f'{height:.2f}%', ha='center', va='bottom', fontweight='bold')

plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.show()

```

===== MODEL COMPARISON TABLE =====

Model Architecture	Test Accuracy (%)	Test Loss	Final Val Accuracy (%)
VGG16	78.0	0.8894	65.12
ResNet50	36.0	1.5445	30.23
MobileNetV2	100.0	0.0157	100.00

